

Assembly Language & Disassembly Primer

Understanding Code Analysis, Memory, and the CPU

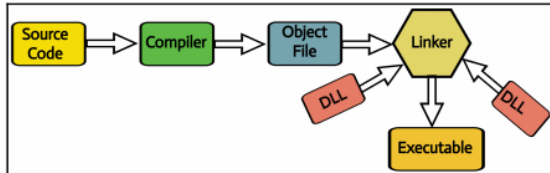
Dr. Manu V.T., NFSU Bhopal

January 29, 2026

From Computer Basics to x64 Architecture

Program Compilation

- The executable compilation process consists of the following steps:
 - ① Source code is written in a high-level language such as C or C++.
 - ② The compiler translates the source code into an intermediate form (object file / machine code), which is not human-readable and is intended for execution by the processor.
 - ③ The linker combines the object code with required libraries (e.g., DLLs) to produce a final executable that can run on the system.

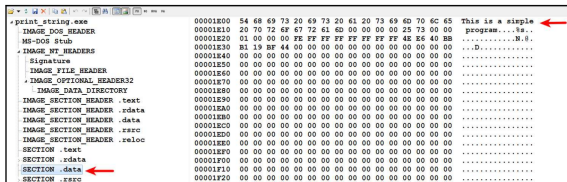


Program On Disk

```
#include <stdio.h>
int main() {
    char *string = "This is a simple program";
    printf("%s", string);
    return 0;
}
```

Executable Sections After Compilation

- The program is compiled into an executable file (print_string.exe).
- Using the **PE Internals** tool, the compiled executable reveals multiple sections generated by the compiler:
 - .text
 - .rdata
 - .data
 - .rsrc
 - .reloc
- For code analysis, the primary focus is on:
 - .text — program code
 - .data — initialized data



```
print_string.exe 00001E00 54 68 69 73 20 69 73 20 61 20 73 69 6D 70 6C 65 This is a simple
IMAGE_DOS_HEADER 00001E10 20 70 72 6F 67 72 61 6D 00 00 00 00 25 73 00 00 program...is..
MS-DOS Stub 00001E20 01 00 00 00 FE FF FF FF FF FF FF 4E E6 40 B8 .....B.B.
IMAGE_NT_HEADERS 00001E30 B1 19 BF 44 00 00 00 00 00 00 00 00 00 00 00 00 ...D.....
Signature 00001E40 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
IMAGE_FILE_HEADER 00001E50 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
IMAGE_OPTIONAL_HEADER32 00001E60 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
IMAGE_DATA_DIRECTORY 00001E70 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
IMAGE_SECTION_HEADER .text 00001E80 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
IMAGE_SECTION_HEADER .rdata 00001E90 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
IMAGE_SECTION_HEADER .data 00001EA0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
IMAGE_SECTION_HEADER .rsrc 00001EB0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
IMAGE_SECTION_HEADER .reloc 00001EC0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
SECTION .text 00001ED0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
SECTION .rdata 00001EE0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
SECTION .data 00001EF0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
SECTION .rsrc 00001F00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

Data Stored in Executable Sections

- The string "This is a simple program" is stored in the .data section at file offset 0x1E00.
- This string is **data**, not executable code, and is required during program execution.
- Other important sections include:
 - .rdata — read-only data and sometimes import/export information
 - .rsrc — resources used by the executable

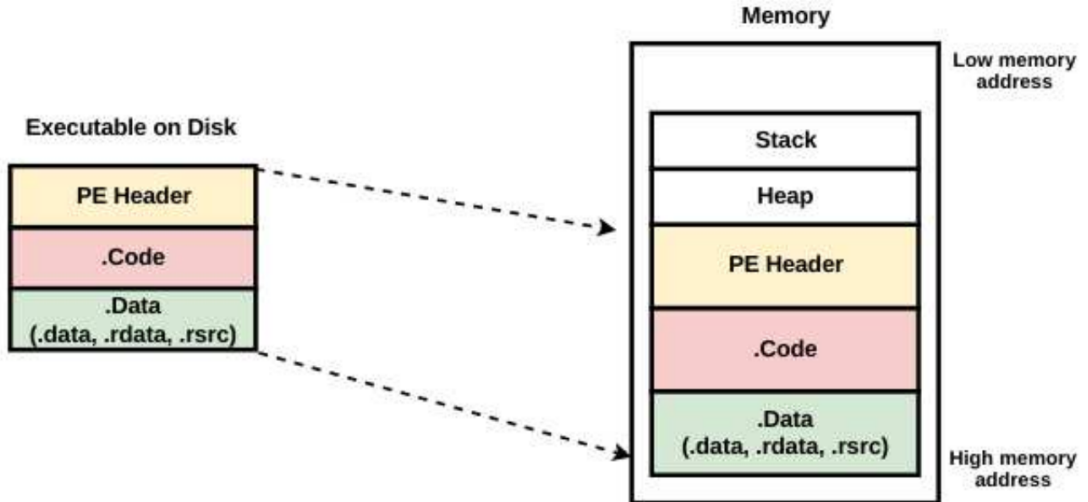
```
print_string.exe 00001E00 54 68 69 73 20 69 73 20 61 20 73 69 6D 70 6C 65 This is a simple
-IMAGE_DOS_HEADER 00001E10 20 70 72 6F 67 72 61 6D 00 00 00 00 25 73 00 00 program....%s..
-MS-DOS Stub 00001E20 01 00 00 00 FE FF FF FF FF FF FF FF 4E E6 40 BB .....N.@.
-IMAGE_NT_HEADERS 00001E30 B1 19 BF 44 00 00 00 00 00 00 00 00 00 00 00 ...D.....
- Signature 00001E40 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
- IMAGE_FILE_HEADER 00001E50 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
- IMAGE_OPTIONAL_HEADER32 00001E60 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
- IMAGE_DATA_DIRECTORY 00001E70 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
- IMAGE_SECTION_HEADER .text 00001E80 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
- IMAGE_SECTION_HEADER .rdata 00001E90 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
- IMAGE_SECTION_HEADER .data 00001EA0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
- IMAGE_SECTION_HEADER .rsrc 00001EB0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
- IMAGE_SECTION_HEADER .reloc 00001EC0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
- SECTION .text 00001ED0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
- SECTION .rdata 00001EE0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
- SECTION .data 00001EF0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
- SECTION .rsrc 00001F00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001F10 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001F20 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
```

Machine Code in the .text Section

- The .text section contains the program's machine code.
- In this example, 35 bytes starting at file offset 0x400 represent executable instructions.
- The compiler translates high-level source code into machine code (machine language).
- Machine code is difficult for humans to read, but directly interpreted and executed by the CPU.
- The compiler separates executable code and data into different sections on disk.

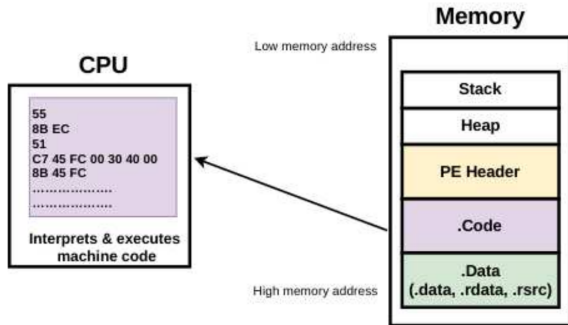
00000400	55 8B EC 51 C7 45 FC 00 30 40 00 8B 45 FC 50 68	U..Q.E..0@..E.Ph
00000410	1C 30 40 00 FF 15 98 20 40 00 83 C4 08 33 C0 8B	.0@.... @....3..
00000420	E5 5D C3 CC FF 25 98 20 40 00 CC CC CC CC CC CC	.]...%. @.....
00000430	55 8B EC E8 38 03 00 00 A3 40 30 40 00 6A 01 FF	U...8....@0@.j..

Program In Memory



Program Execution in Memory

- After loading the executable into memory, the CPU begins execution.
- The CPU fetches machine instructions from memory, decodes them, and executes them.
- During instruction execution, required data is also fetched from memory.
- In this example:
 - Instructions are fetched from the `.text` section.
 - The string "This is a simple program" is fetched from the `.data` section.



Disassemblers and Debuggers

- Assembly language is human-readable and represents CPU instructions.
- Analyzing assembly code helps determine a program's functionality and internal behavior.
- From a code analysis perspective, determining the program's functionality mainly relies on understanding these assembly instructions and how to interpret them.

010F1000	55	push ebp	
010F1001	8B EC	mov ebp,esp	
010F1003	51	push ecx	
010F1004	C7 45 FC 00 30 0F 01	mov dword ptr ss:[ebp-4],print_string.10F3000	10F3000:"This is a simple program"
010F100B	8B 45 FC	mov eax,dword ptr ss:[ebp-4]	
010F100E	50	push eax	
010F100F	68 1C 30 0F 01	push print_string.10F301C	10F301C: "%s"
010F1014	FF 15 98 20 0F 01	call dword ptr ds:[<&printf>]	
010F101A	83 C4 08	add esp,8	
010F101D	33 C0	xor eax,eax	
010F101F	8B E5	mov esp,ebp	
010F1021	5D	pop ebp	
010F1022	C3	ret	

Hexadecimal Bytes and Machine Code

The hexadecimal values you see are **raw machine code bytes**. In assembly language terms, they represent **encoded CPU instructions and operands** that the processor executes directly.

How to read this

Each hex byte (00–FF) corresponds to one of the following:

- **Opcode** — the instruction itself
- **Operands** — registers, constants, or memory addresses
- **Instruction prefixes / metadata**

Assembly language is the human-readable representation of these bytes after decoding.

Machine Code & Opcodes

Assembly is just a mnemonic representation of raw machine code bytes.

- **Opcode:** The operation code byte(s) telling the CPU what to do.
- **Operands:** The data or addresses following the opcode.

Disassembly View

Address	Bytes	Instruction
00401000	B8 05 00 00 00	mov eax, 5
00401005	83 C0 03	add eax, 3

Note: B8 is the opcode for "move immediate into EAX".

What Is an Opcode?

- An **opcode** is a byte (or bytes) that specifies a CPU instruction.
- Opcodes are defined by the processor architecture (x86, x64, ARM).
- The CPU decodes opcodes using fixed hardware logic.

Why 55 Is an Opcode

- The x86 Instruction Set Architecture (ISA) defines opcode meanings.
- According to the Intel Software Developer's Manual:
 - Opcode range 50-57 = PUSH r32
 - Register is encoded directly in the opcode

How the CPU Interprets 55

- CPU fetches byte at address pointed to by EIP.
- Instruction decoder matches 0x55 to a PUSH instruction.
- The meaning is hardwired into the CPU.
- Software cannot redefine opcode meanings.
- ISA rules & official opcode tables

Fundamental Units

- **Bit:** The smallest unit (0 or 1).
- **Byte:** A group of 8 bits. Represented by two hex digits (e.g., 5D).
- **Nibble:** 4 bits (one hex digit).

Data Sizes

- **Word:** 2 bytes (16 bits). Ex: 8B EC
- **Double Word (Dword):** 4 bytes (32 bits).
- **Quad Word (Qword):** 8 bytes (64 bits).

Little-Endian Format Explained

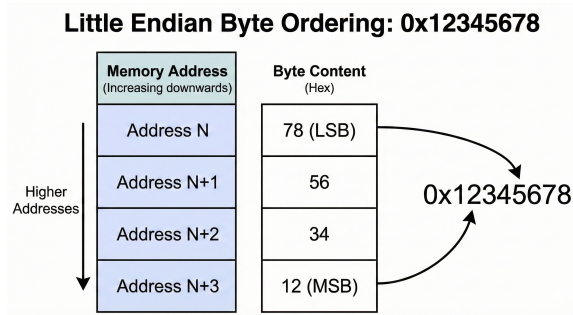
Endianness refers to the order in which bytes of a multi-byte word are stored in memory. x86 architecture is **Little-Endian**.

Example: Storing 0x12345678

Consider the 32-bit value 0x12345678 stored at address 0x4000.

Offset	Byte Stored	Significance
0x4000	78	Least Significant Byte (LSB)
0x4001	56	
0x4002	34	
0x4003	12	Most Significant Byte (MSB)

Key Takeaway: In memory views (Hex Editors), bytes appear reversed compared to the register value.



Memory Organization

Main Memory (RAM)

- An array of bytes where code and data reside.
- Each byte has a unique address.

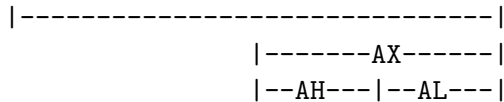
Little-Endian Format

- Data is stored in reverse byte order.
- Least significant byte (LSB) at lowest address.
- Example: $0x12345678 \rightarrow 78\ 56\ 34\ 12$.

CPU Registers (x86)

High-speed storage locations directly inside the CPU.

EAX (32-bit Accumulator)



- **General Purpose:** EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP.
- **EIP:** Instruction Pointer (Next instruction).
- **EFLAGS:** Status flags (Zero, Carry, Overflow).

The Data Registers (A, B, C, D)

These are the workhorses for arithmetic and data movement.

EAX (Accumulator) Used for arithmetic operations.

- **Crucial Relevance:** It stores the **return value of a function**. Check this register immediately after a CALL instruction to see if the function succeeded (e.g., returns 1) or failed (e.g., returns 0).

EBX (Base) Often used as a general base pointer to data locations in memory.

ECX (Counter) Used specifically as a loop counter.

- **Relevance:** Instructions like LOOP automatically decrement ECX. Malware frequently uses ECX for decryption loops or iterating through system structures.

EDX (Data) Used for I/O operations and assisting EAX in arithmetic involving large numbers (multiplication/division).

The Index Registers (Source & Destination)

These registers are primarily used for highly optimized memory operations, particularly dealing with strings or arrays.

String Operations

Malware often uses specific instructions like `MOVS` (Move String) or `STOS` (Store String) which rely implicitly on these two registers.

ESI (Source Index) The pointer to the **source** data location during string or memory copying operations.

EDI (Dest. Index) The pointer to the **destination** data location during string or memory copying operations.

The Critical Pointer Registers (Stack Management)

These two registers are critical for understanding program flow and stack-based vulnerabilities. They track the program's "Stack Frame."

Warning

Corruption of these registers, especially EBP, usually leads to an immediate crash or control-flow hijacking.

ESP (Stack Pointer) Points to the very top of the stack (the lowest address in current use).

- Its value changes constantly with every PUSH and POP instruction.

EBP (Base Pointer) Acts as a stable "anchor" for the current function's stack frame.

- Local variables are accessed as negative offsets ($[EBP-4]$).
- Function arguments are accessed as positive offsets ($[EBP+8]$).

The EFLAGS Register in Detail

The **EFLAGS** register is crucial for control flow. It tracks the result of the previous calculation.

- **ZF (Zero Flag)**: Set to 1 if the result is zero. (Used by `je/jz`).
- **SF (Sign Flag)**: Set to 1 if the result is negative (MSB is 1).
- **CF (Carry Flag)**: Set on unsigned arithmetic overflow.
- **OF (Overflow Flag)**: Set on signed arithmetic overflow.

Implicit Updates

Instructions like `cmp eax, ebx` perform a subtraction ($\text{eax} - \text{ebx}$) purely to update these flags, discarding the result.

Data Transfer Instructions

- **MOV Instruction:** The most fundamental instruction. Moves data from source to destination.

```
mov dst, src
```

- **Register to Register:**

```
mov eax, ebx ; Copy value of EBX into EAX
```

- **Memory to Register:**

```
mov eax, [0x403000] ; Load value at address into EAX
```

- **Immediate Values:**

```
mov eax, 10 ; Set EAX to decimal 10
```

Arithmetic & Bitwise Operations

Math Operations

```
add  eax, 5      ; Add
sub   ebx, eax    ; Subtract
inc   eax        ; Increment
div   ebx        ; Divide
```

Bitwise Logic

```
and  eax, ebx    ; Bitwise AND
xor  eax, eax    ; Clear Register
shl  eax, 2      ; Shift Left
```

Branching & Conditionals

Assembly uses **Jumps** to handle control flow (If/Else, Loops).

1. Compare:

```
cmp eax, 0 ; Sets EFLAGS
```

2. Jump:

```
jz label ; Jump if Zero flag is set
```

Common Jumps:

- jmp: Unconditional jump.
- je / jz: Jump if Equal / Zero.
- jne / jnz: Jump if Not Equal / Not Zero.

Implementing Loops

C Code

```
int i = 0;
while (i < 5) {
    i++;
}
```

Assembly Translation

```
mov [i], 0      ; Init
start_loop:
cmp [i], 5      ; Condition
jge end_loop    ; Exit if >= 5
add [i], 1      ; Body
jmp start_loop  ; Repeat
end_loop:
```

Arrays & Strings

- **Contiguous Memory:** Arrays are stored as a block.
 - `int nums[3]` takes 12 bytes (3×4 bytes).
- **Access Calculation:**

$$\text{Address} = \text{Base} + (\text{Index} \times \text{ElementSize})$$

- **Assembly Access:**

```
mov eax, [ebx + ecx*4]  
; EBX=Base, ECX=Index, 4=int size
```

- **Strings:** Just arrays of characters (1 byte each). Often null-terminated (0x00).